Deep Dive MCP Server Protocol JSON-RPC Communication Analysis

A 25-minute tour of the Model Context Protocol — mostly live demo, with these pages as the map.

Explore the MCP Communication Flow

We build up one real MCP exchange — discovery, resources, tools, prompts, then the newer pieces: round-trip requests, MCP Apps, tasks, and finally a supervised agent driving the same tool. Each page is one lens on the same JSON-RPC traffic.

Format: this is a show-and-tell. Roughly half the time is live in the Inspector and the IDE — the slides stay deliberately thin, and the Trace Log and Runtime Facts pages are the reference to come back to afterwards.

25-Minute Run Sheet

Slides ~11 min, live demo ~13 min, wrap 1 min.

Time	Segment	Point to land
0 – 2	Why MCP	One protocol instead of N bespoke integrations
2 – 5	Overview	Four actors, one round trip — and no session
5 – 8	Flow Diagram	It is just JSON-RPC over stdio — hover for real payloads
8 – 13	DEMO Inspector	Discover, then resources / tools / prompts live
13 – 15	Extensions	The server asks a question by answering the call
15 – 18	MCP Apps	A tool can return a UI, not just text
18 – 21	Tasks	Call now, fetch later — long work stops blocking
21 – 24	DEMO Agent	Same tool, now inside a supervised audit
24 – 25	Wrap	Where the code and lessons live

Protocol Overview 2 – 5

High-level architecture of the Model Context Protocol with an interactive Mermaid sequence diagram. How users, applications, LLMs, and MCP servers fit together in one request-response cycle.

Flow Diagram 5 – 8

The whole exchange as one numbered sequence diagram — `server/discover`, cacheable reads, a round-trip tool call, a task handle, and a subscription that closes at once. Hover any message to see the exact JSON that crossed the wire.

Extensions & Round Trips new

How the protocol grows without breaking clients, via namespaced `extensions` keys — and Multi Round-Trip Requests, where a server that can no longer send requests asks which directory to search by answering the call.

MCP Apps new

Tool + `ui://` resource + one `_meta` field (written under two keys) = an interactive dashboard rendered inside the conversation. No new methods, and a sandboxed iframe keeps it safe.

Tasks 2026-07-28

The tasks extension — call now, fetch later, and now the server's decision rather than a client opt-in. The extension shape, the lifecycle state machine, and what is left after `tasks/result` and `tasks/list` were removed.

Supervised Agent new

The payoff. Four plugin skills wrap the same MCP tool into a codebase audit that finds the hotspot, explains why, and pauses for approval before writing anything.

Runtime Facts

Reference card: the protocol revision, the per-request `_meta` envelope, every capability declared, the new error codes, the removed methods, and the tool, prompt, and resource inventory.

Trace Log

Raw, timestamped JSON-RPC messages exactly as they were exchanged, with filters for sent / received and a JSON pretty-printer. The ground truth behind every other page.

About This Traffic

Everything on these pages comes from real captured traffic:

- **Client:** inspector-cli v2.5.0 (MCP Inspector)
- **Server:** agent-mcp-workshop v0.0.1 (Java, stdio transport)
- **Protocol:** MCP 2026-07-28 — stateless: no handshake, no session
- **Duration:** ~50 seconds (17:55:28 – 17:56:16)

In under a minute it exercises:

- **Discovery:** `server/discover` — supported revisions, capabilities, and identity, with no session created
- **Reads:** 60+ Javadoc resources, prompts, and tools — all cacheable, so all carrying `ttlMs` and `cacheScope`
- **Round trips:** a `tools/call` answered with `input_required`, then re-sent by the client with the answer
- **MCP Apps:** a tool advertising `_meta.ui.resourceUri` plus its `ui://` HTML resource
- **Tasks:** a task handle the server chose to return, and polling to completion — no status notification, because nothing subscribed
- **Rejections:** the removed `initialize` at `-32601`, plus `-32022` and `-32602` for a bad envelope

Server capabilities advertised: `completions`, `prompts`, `resources`, `tools`, and `extensions` (`io.modelcontextprotocol/ui`, `io.modelcontextprotocol/tasks`). Client side, repeated on every single request: `elicitation` and the same two extensions — plus a deprecated `roots` that real clients still send and this server ignores.

© 2026 MCP Workshop. Multi-page analysis of JSON-RPC communication flow.

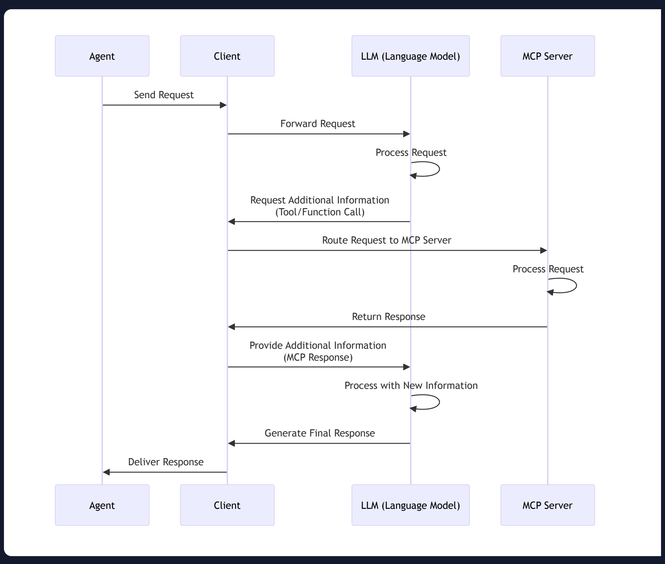
MCP Protocol Overview

2 – 5 min

Understanding the Model Context Protocol architecture and communication flow — revision 2026-07-28

MCP Communication Architecture

The Model Context Protocol (MCP) enables seamless communication between applications, language models, and external services. The diagram below illustrates the complete request-response flow in an MCP-enabled system.



Flow Explanation

1. Agent Interaction

The process begins when an `Agent` sends a request to the `Client`. This could be a question, command, or any input that requires processing.

2. LLM Processing

The `Client` forwards the request to the `LLM (Language Model)`, which begins processing the agent's input. During this processing, the LLM may determine that it needs additional information or capabilities to provide a complete response.

3. Tool/Function Call

When the `LLM` requires external data or functionality, it makes a `Tool/Function Call` back to the Client. This is where MCP comes into play - the LLM is requesting access to external resources, data, or capabilities.

4. MCP Server Interaction

The `Client` routes this request to the appropriate `MCP Server`. The MCP Server processes the request, which might involve:

- Accessing databases or file systems
- Calling external APIs
- Performing computations
- Retrieving specific resources or data

5. Response Chain

The `MCP Server` returns its response to the Client, which then provides this additional information back to the LLM. The LLM processes this new information along with the original request to generate a comprehensive final response.

6. Final Delivery

The `Client` delivers the LLM's final response back to the `Agent`, completing the request-response cycle.

Key Benefits of MCP Architecture

- **Modularity:** MCP servers can be developed and deployed independently
- **Extensibility:** New capabilities can be added without modifying the core application
- **Security:** Controlled access to external resources through well-defined protocols
- **Standardization:** Consistent interface for LLM-to-external-service communication
- **Scalability:** Multiple MCP servers can handle different types of requests

Protocol Features

The MCP protocol supports various types of interactions:

- **Tools:** Function calls that the LLM can invoke
- **Resources:** Data sources that can be read or accessed
- **Prompts:** Reusable prompt templates
- **Sampling:** AI model interaction capabilities — deprecated on `2026-07-28`, so this server does not implement it
- **Real-time Updates:** Dynamic notifications of changes

Three newer pieces build on those foundations, and this server implements all three:

- **Round trips:** the server asks the user a question by *answering* the call, then the client re-sends it with the answer
- **MCP Apps:** a tool returns an interactive UI instead of a wall of text
- **Tasks:** call now, fetch later — long-running work stops blocking

One Thing to Keep in Mind: It Is Stateless

Revision `2026-07-28` removed the `initialize` handshake, and with it the whole notion of a session. That single change explains almost everything else you will see today.

- **Discovery replaces the handshake.** `server/discover` reports which revisions a server speaks, what it can do, and who it is — and creates nothing.
- **Every request re-states the lifecycle.** The protocol version and the client's capabilities travel in `params._meta` on each call, because the server cannot remember them.
- **The server can no longer call the client.** Modern clients discard inbound requests, so questions are embedded in a result and the client re-sends the original request with the answer.
- **State that has to survive travels with the message.** That is what `requestState` is for — there is nowhere on the server to put it.

Everything in this deck is one consequence or another of that removal — see the Flow Diagram for the shape it produces on the wire.

🔍 Try It Yourself

Want to see this diagram in action? The **MCP Inspector** is the perfect tool for exploring and debugging MCP server communications in real-time.

🚀 MCP Inspector

Interactive tool for testing and debugging MCP servers - the same tool used to generate the communication traces in this workshop!

View on GitHub →

The Inspector allows you to:

- Connect to any MCP server and explore its capabilities
- Test tools, resources, and prompts interactively
- View real-time JSON-RPC message exchanges
- Debug server implementations and protocol compliance
- Generate communication traces like the ones analyzed in this workshop

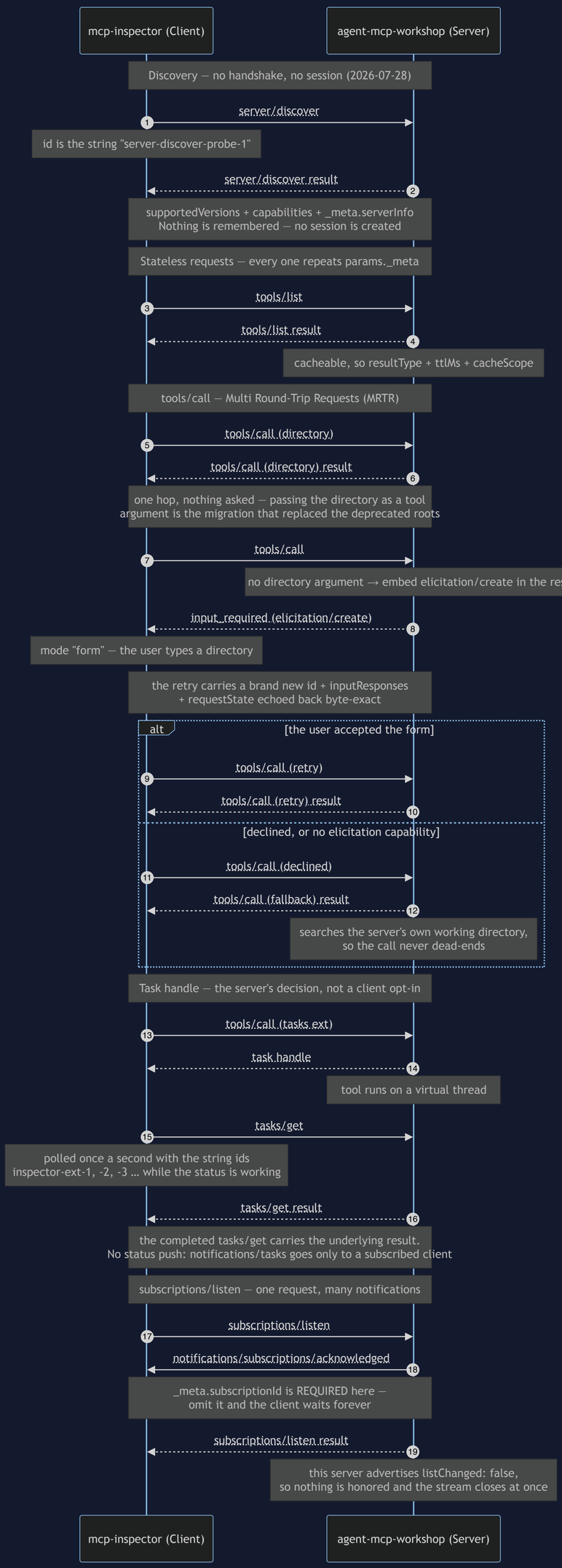
Pro tip: Use the Inspector to connect to the agent-mcp-workshop server and recreate the exact communication flow shown in our diagrams!

© 2026 MCP Workshop. Protocol overview and architecture explanation.

MCP Workshop – JSON-RPC Communication Flow

5 – 8 min

The whole exchange as one sequence diagram — discovery, cacheable reads, a round-trip tool call, a task handle, and a subscription that closes at once. Hover any message for the real JSON.



Tip: Hover or tap to scroll the diagram if it overflows. This diagram shows the full stateless lifecycle: discovery, cacheable reads, an MRTR round trip, a task handle, and a subscription that closes at once.

© 2026 MCP Workshop. Diagram rendered with Mermaid.

Extensions & Round Trips

13 – 15 min

How the protocol grows without breaking — and how a server asks a question when it can no longer send requests.

Extensions in One Sentence

Anything beyond the core protocol lives under `capabilities.extensions` as a namespaced key. A peer that does not recognise the key simply ignores it — so new features ship without breaking old clients. (`capabilities.experimental` survives for non-standard, in-house capabilities.)

What this server declares

```
"extensions": {
  "io.modelcontextprotocol/ui": {},
  "io.modelcontextprotocol/tasks": {}
}
```

Empty means no settings. `mimeTypes` is a client setting, sent in each request's envelope.

Identifier format

`{vendor-prefix}/{extension-name}`

- Official: `io.modelcontextprotocol/ui`
- Yours: `com.acme/audit-trail`

Clients declare theirs the same way, inside every request's `_meta` .

The Server Can No Longer Call You

On `2026-07-28` a server MUST NOT send a request to a client — modern clients silently discard inbound requests. That kills the three things servers used to ask for: `roots/list` , `sampling/createMessage` , and `elicitation/create` . All three survive as *embedded* requests; two of them, roots and sampling, are also deprecated under SEP-2577, so this server builds only `elicitation/create` .

Multi Round-Trip Requests (MRTR): the server asks by *answering*. It replies `resultType: "input_required"` with the question embedded in the result; the client gathers the answers and **re-sends the original request as a brand new request with a brand new id**, carrying `inputResponses` and the `requestState` echoed back byte-exact.

An embedded request

```
"inputRequests": {
  "search_directory": {
    "method": "elicitation/create",
    "params": { "mode": "form" }
  }
}
```

A keyed **map**, not an array. Each entry has `method` plus optional `params` — and no `jsonrpc` , no `id` . Only those three methods may be embedded, and this server builds just this one. (`params` abbreviated; the full form is below.)

Why `requestState` exists

Sessions are gone, so the server has nowhere to remember what it was doing. The continuation travels to the client and back:

```
"requestState": "eyJzdGFnZSI6ImRpcmVjdG9yeSJ9"

  decodes to
  {"stage":"directory"}
```

An **opaque string** to the client. This repo happens to base64 its JSON; the protocol only guarantees the exact bytes come back.

Elicitation — The One Question Left

The `key_word_search` tool needs a directory, and there are two ways it can get one. Pass `arguments.directory` and the call finishes in a single hop — that argument *is* the migration the specification names in place of the now-deprecated roots. Omit it — which is what a caller working from the tool description alone usually does — and the server has to ask, so it embeds a form.

1. `tools/call` arrives with no `directory` argument, so the server answers `input_required` embedding `elicitation/create` .
2. The client renders the form (`mode: "form"`) and the user types a path.
3. Client re-sends `tools/call` with a **new id**, the same `requestState` , and the answer under `inputResponses.search_directory` .
4. Server decodes its own `requestState` , reads the directory out of the map, searches, and returns `resultType: "complete"` . It never stored a thing.
5. Declined the form, or could not show one? The server stops asking and searches its own working directory — the call always returns a result, never a dead end. Not `-32021` either: that code is for a capability the request cannot be served without, and this one can be served.

There used to be a cheaper question ahead of that form: an embedded `roots/list` , which a client answers from its own configuration without troubling anyone, escalating to the form only when the roots came back empty. Roots is deprecated under SEP-2577, so that stage is gone and the `directory` argument took its place. Deprecated is not removed, though — roots stays in the spec and works for at least twelve months, and real clients (the Inspector included) still declare `"roots": {"listChanged": true}` on every request. This server simply does not read the key.

Always leave a one-hop path. MRTR is optional for clients, and not every one implements it — the Inspector's own task-augmented `tools/call` path rejects `input_required` outright. A tool whose only route to an answer is a round trip is a tool those callers cannot use at all, so take the input as an optional argument and treat asking as the fallback.

The embedded elicitation

```
{
  "resultType": "input_required",
  "inputRequests": {
    "search_directory": {
      "method": "elicitation/create",
      "params": {
        "mode": "form",
        "message": "Pick a directory for the
           keyword-search tool to search in:",
        "requestedSchema": {
          "type": "object",
          "properties": { "directory": {
            "type": "string",
            "title": "Search Directory" } },
          "required": ["directory"]
        }
      }
    }
  },
  "requestState": "eyJzdGFnZSI6ImRpcmVjdG9yeSJ9"
}
```

New: `mode` is `"form"` or `"url"` . Gone: the old `elicitationId` field — the retry correlates the answer now.

The answer, on the retry

```
{
  "method": "tools/call",
  "id": 3,
  "params": {
    "_meta": { ...envelope... },
    "name": "key_word_search",
    "arguments": { "keyword": "record" },
    "requestState": "eyJzdGFnZSI6ImRpcmVjdG9yeSJ9",
    "inputResponses": {
      "search_directory": {
        "action": "accept",
        "content": {
          "directory": "/Users/.../mcp/tools"
        }
      }
    }
  }
}
```

An input response is the bare result the method would have returned — nothing JSON-RPC-shaped. Three actions: `accept` , `decline` , `cancel` ; only `accept` carries content.

Request IDs Are Strings *or* Numbers

With server-initiated requests gone, so is the reserved negative-id range that used to mark them. What replaced it is a subtler trap: the client picks the id, and it is not always a number. A server MUST echo it back in exactly the form it arrived.

Request	Id as sent
`server/discover`	`"server-discover-probe-1"`
`subscriptions/listen`	`"listen:0"`
`tasks/get` (polling)	`"inspector-ext-1"` , `"inspector-ext-2"` …
everything else	`1` , `2` , `3` …

This is why `spec/RequestId.java` and `spec/RequestIdTypeAdapter.java` exist. Before the migration `id` was a `Long` , and the very first message of the new revision crashed the server with `NumberFormatException: For input string: "server-discover-probe-1"` .

Live Demo

- Call `key_word_search` with a `directory` argument first — one request, one `complete` result. That is the baseline.
- Drop the argument and call it again — now the reply is `input_required` , neither an answer nor an error.
- Point at the `requestState` , then at the **new id** on the retry: two JSON-RPC exchanges, one logical call.
- Decline the form and run it again — the search still returns, rooted at the server's working directory. There is no dead end to demo, by design.

© 2026 MCP Workshop. Extensions and Multi Round-Trip Requests — quick overview.

MCP Apps

15 – 18 min

A tool that returns a UI, not just text — rendered inline in the conversation.

The Problem

Without MCP Apps

The tool returns a line of `path, keyword_count=N` per file — 102 of them for `mcp` across this repository's own source. The user reads a wall of text and cannot sort, filter, or click anything.

With MCP Apps

A page renders beside the result inside the chat, and a full app can re-run the search from the panel itself. The workshop's page shows sample rows, with those calls written out in its comments.

The Whole Pattern

Tool `key_word_search` + **Resource** `ui://` HTML + **_meta** links the two = **Interactive App** rendered in the conversation

No new methods. MCP Apps is `tools/list` and `resources/read` doing exactly what they already did — plus one metadata field. Both of those are cacheable methods now, so their results also carry `ttlMs` and `cacheScope` : the host can preload the HTML once and reuse it.

Three Changes to the Server

1. Declare the extension in `server/discover`

```
"extensions": {
  "io.modelcontextprotocol/ui": {}
}
```

Empty: the extension's one setting, `mimeTypes` , is the host's, sent in each request's client capabilities.

2. Point the tool at its UI via `_meta` — under *both* keys

```
{
  "name": "key_word_search",
  "inputSchema": { ... },
  "_meta": {
    "ui": { "resourceUri": "ui://keyword-search/mcp-app.html" },
    "ui/resourceUri": "ui://keyword-search/mcp-app.html"
  }
}
```

The nested `ui.resourceUri` and the flat `ui/resourceUri` hold the same value. The nested one is canonical; the flat one is **deprecated** and will be removed before GA. Hosts read one or the other depending on their vintage, so `AppMeta.of(...)` mirrors the value into both.

3. Serve that URI as HTML with the app MIME type

```
ui://keyword-search/mcp-app.html
  → text/html;profile=mcp-app
```

In this codebase: `AppTool` , `AppMeta` , and `UiMeta` carry the metadata, `AppToolBuilder.withResourceUri(...)` wires it up, and `resources/read` answers the page's `ui://` address with `lesson/mcp-app.html` from the classpath.

One Field That Went Away

`Tool.execution` — and with it the `taskSupport` hint — was removed from `tools/list` on `2026-07-28` . Clients strip it silently, so a server still emitting it just wastes bytes.

Gone

```
"execution": {
  "taskSupport": "optional"
}
```

Why

The hint forced a `tools/list` warmup before any call could be made as a task. Whether a call becomes a task is now decided by the server, per request — see Tasks.

What Happens at Runtime

1. Host calls `tools/list` and sees `_meta.ui.resourceUri` (or the flat `_meta["ui/resourceUri"]`).
2. Host preloads the HTML with `resources/read` — before the tool is ever called.
3. User asks a question; host calls `tools/call` as normal.
4. Host renders the preloaded HTML in a sandboxed iframe and hands it the tool result over `postMessage` .
5. A full app can call the tool again — the user re-searches without typing a new prompt. The workshop's page stops at its sample rows.

Security Model

The UI is untrusted content, so the host sandboxes it:

- No direct network access
- No access to the host page or DOM
- No credentials or tokens

The only channel out is a controlled `postMessage` bridge — the app may request tool calls, but the host decides whether to run them.

Live Demo

- `resources/list` — spot `ui://keyword-search/mcp-app.html` , named *Keyword Search App*, among the Javadoc resources.
- `resources/read` it and show the returned MIME type `text/html;profile=mcp-app` .
- In the Inspector's **Apps** tab, pick `key_word_search` under **MCP Apps (1)**, give it a keyword and a directory, and click **Open App**. The page renders with its sample rows; the real result is the `tools/call` reply in the Protocol panel.

© 2026 MCP Workshop. MCP Apps (io.modelcontextprotocol/ui) — quick overview.

MCP Tasks — Call Now, Fetch Later

2026-07-28 extension · 18 – 21 min

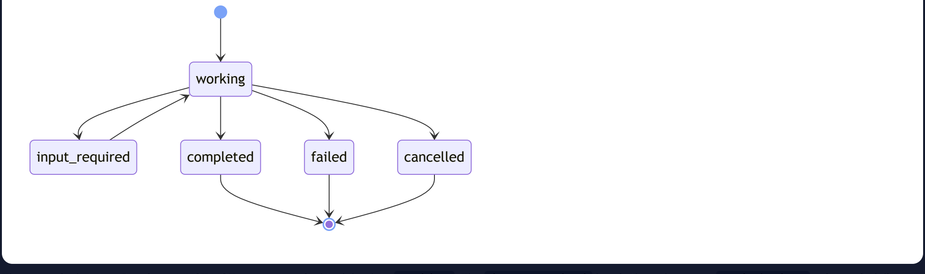
A quick tour of the tasks extension, at the Beta stage and headed for core — long work stops blocking, and the server decides when

The Idea in One Sentence

Instead of holding a JSON-RPC request open until the work completes, the server immediately replies with a **task handle** (`resultType: "task"`) and runs the work in the background. The client polls `tasks/get` on its own schedule; once the status is terminal, that same poll carries the underlying result.

Now the server's decision. The old per-request `params.task` opt-in is gone, and servers MUST ignore it if one arrives. A handle may only be handed to a client that declared the tasks extension *on the very request being answered* — there is no session in which to remember it.

The Lifecycle



Three terminal states. Receivers MUST NOT move a terminal task back to `working` . The `input_required` hop is the only reason `tasks/update` exists — it delivers `inputResponses` to a task that is waiting.

An Extension Now, Not Core

The top-level `tasks` capability slot is gone. Both sides declare the extension by identifier instead — and the client re-declares it on every request, because there is no handshake to declare it once.

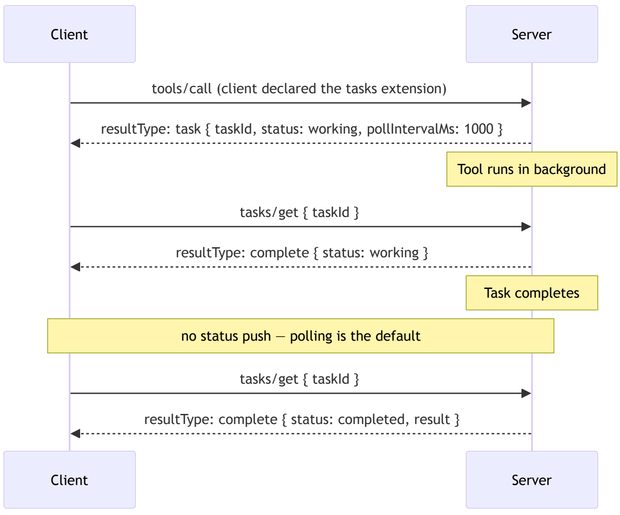
Server, in `server/discover`

```
"capabilities": {
  "tools": { "listChanged": false },
  "extensions": {
    "io.modelcontextprotocol/tasks": {}
  }
}
```

Client, in every `params._meta`

```
"io.modelcontextprotocol/clientCapabilities": {
  "extensions": {
    "io.modelcontextprotocol/tasks": {}
  }
}
```

The Wire Flow



A server may push status only to a client that subscribed through `subscriptions/listen` ; this one pushes none, so the client polls. When a server does push, the notification is the bare `notifications/tasks` — the `notifications/tasks/` prefix is reserved, so `notifications/tasks/status` is not a legal name. Its params are the whole task snapshot.

What Is Left, and What Went

Method	Purpose
`tasks/get`	Poll status; carries `result` , `error` , or `inputRequests` as the status calls for
`tasks/update`	Deliver `inputResponses` to a task sitting in `input_required`
`tasks/cancel`	Cooperative cancel, acknowledged with an empty result — a finished task keeps its status; `-32602` only for an unknown `taskId`
`notifications/tasks`	Optional status push, only to a client subscribed through `subscriptions/listen` . This server sends none

Removed: `tasks/result` (clients poll `tasks/get` instead) and `tasks/list` (with no session to scope a listing to, it would leak other callers' task ids). Both now answer `-32601` . **Renamed:** `ttl` → `ttlMs` and `pollInterval` → `pollIntervalMs` .

A handle or a question, never both. `resultType` holds one value, so a `tools/call` answers *either* `task` *or* `input_required` . Commit to the shape before the tool works out what it is missing — answer a question to a task-declaring client and the Inspector rejects it with *"Unsupported result type `input_required` for `tools/call` "*. A task asks later, as a *status*, through `tasks/update` .

Live Demo

- Call `key_word_search` from a client declaring the tasks extension — the reply is a handle, not the answer. Nothing in the request asked for that.
- Watch the Inspector poll `tasks/get` once a second with string ids `inspector-ext-1` , `-2` , `-3` … until `status` flips.
- Then run `inspector/modern-tasks.sh` : its `tasks/cancel` of the finished task is acknowledged, and the poll after it still says `completed` — terminal is terminal. `tasks/list` and `tasks/result` are gone, and answer `-32601` .

© 2026 MCP Workshop. Tasks extension (io.modelcontextprotocol/tasks, MCP 2026-07-28).

From Server to Supervised Agent

21 – 24 min

The payoff: the tool we built becomes one layer of an agent that reasons about the answer.

Why This Matters

The Inspector proves the protocol works. It does not show *value* — a keyword count is not an insight. Wrapping the same MCP tool in a plugin of skills turns "deprecated: 7 occurrences across 5 files" into "the router is where this revision's deprecations land, and here is why."

> **Supervised agent:** the agent does the tedious work — searching, analysing, cross-referencing — then **stops and asks** before producing the consequential output. You stay the decision maker.

Four Skills, Three Layers

DATA
keyword-search
Calls the `key_word_search` MCP tool and sorts files by count.

ANALYSIS
keyword-report
Groups hits by package, finds concentration, writes a markdown report.

EXECUTION
java-script-runner
Runs a single `.java` file with no build step (JEP 330) — ships an embedded `FileAnalyzer`.

ORCHESTRATOR
audit
Drives the other three, reasons about the hotspot, pauses at the checkpoint.

Layer	Who does it	Good at
Data	MCP tool (Java server)	Fast, deterministic file scanning
Execution	JVM via single-file source	Real structural analysis, no build
Reasoning	The model	Explaining *why*, prioritising the fix

The Audit Workflow

`/keyword-audit-plugin:audit` — or with your own list: `keywords=[TODO,FIXME,HACK,deprecated]`

1. **Search** — `key_word_search` once per keyword, via MCP.
2. **Identify** — the file with the highest single-keyword count is the hotspot.
3. **Analyze** — run `FileAnalyzer` on the hotspot through the JVM.
4. **Read + reason** — read the source and explain why *that* file leads.
5. **Report** — per-keyword breakdown with concentration figures.

> ⏸ **Checkpoint** — "Proceed to synthesis? (yes/no)" — nothing is written until you say yes.

6. **Synthesis** — health score, prioritised actions, then offers to save `audit-report.md`.

Skill vs Plugin

A skill is one folder

```
skills/keyword-search/
  SKILL.md    ← name, description,
                instructions
```

A plugin bundles skills

```
keyword-audit-plugin/
  .claude-plugin/plugin.json
  skills/
    keyword-search/  keyword-report/
    java-script-runner/  audit/
```

Skills are plain markdown — the interesting engineering is the MCP server underneath. Launch both together with `./start_claude_mcp.sh`, which builds the JAR, resolves `mcp.json` into `.mcp-resolved.json`, and passes `--plugin-dir`.

Live Demo

- `./start_claude_mcp.sh` — server plus plugin in one command.
- Run `/keyword-audit-plugin:audit` and let it work.
- Stop at the checkpoint — that pause is the whole point.

© 2026 MCP Workshop. Supervised agent over the workshop MCP server.

MCP Workshop – Runtime Facts

Reference card for the captured traffic: the revision, the per-request envelope, declared capabilities, and what the server exposes.

Runtime Facts — MCP 2026-07-28, matching the Trace Log

Protocol Information

- **Protocol Revision:** 2026-07-28
- **Advertised as:** `supportedVersions: ["2026-07-28"]`
- **JSON-RPC Version:** 2.0
- **Transport:** stdio
- **Lifecycle:** stateless — no handshake, no session

Client Details

- **Name:** inspector-cli
- **Version:** 2.5.0
- **Role:** MCP Client
- **Declared per request** in `_meta`, not once at startup

Server Details

- **Name:** agent-mcp-workshop
- **Version:** 0.0.1
- **Role:** MCP Server (Java)
- **Identity travels in** `_meta["io.modelcontextprotocol/serverInfo"]`

The Request Envelope

Carried in `params._meta` on every single request:

- **Required:** `io.modelcontextprotocol/protocolVersion`
- **Required:** `io.modelcontextprotocol/clientCapabilitie`
- **Recommended:** `io.modelcontextprotocol/clientInfo`
- **Optional:** `io.modelcontextprotocol/logLevel` — absent means the server MUST stay silent on `notifications/message`
- **Optional:** `traceparent`, `tracestate`, `baggage` — W3C trace context, reserved *unprefixed* by explicit exception so OpenTelemetry libraries work unchanged

Missing a required field is `-32602`. This is what replaced the handshake.

Client Capabilities

Per-request facts, not session state — a client may send a different set on each call, and the server reads it fresh every time. It acts on exactly one of them, plus the extensions map:

- **Elicitation:** ✓ (form, url) — the one capability this server reads and acts on
- **Extensions:** `io.modelcontextprotocol/tasks`, `io.modelcontextprotocol/ui`
- **Roots:** still arrives as `listChanged: true`; deprecated under SEP-2577, modelled nowhere here, and ignored as an unrecognised key
- **Sampling:** same story — clients still declare it, this server never asks for it

Clients are *supposed* to keep declaring the deprecated two throughout the deprecation period, so their arrival is correct behaviour, not a client bug.

Server Capabilities

Exactly seven optional slots, all of them: `experimental`, `logging`, `completions`, `prompts`, `resources`, `tools`, `extensions`. This server sets five.

- **Completions:** ✓
- **Prompts:** ✓ (listChanged: false)
- **Resources:** ✓ (listChanged: false, subscribe: false)
- **Tools:** ✓ (listChanged: false)
- **Extensions:** ✓ (see next card)

Extensions

- `io.modelcontextprotocol/ui` — `{}`
- `io.modelcontextprotocol/tasks` — `{}`

Namespaced keys under `capabilities.extensions` — the old top-level `tasks` slot is gone, and unknown keys are ignored. MCP Apps defines no server settings: `mimeTypes` is what a *client* declares. See Extensions.

Result Shape

- `resultType` **on every result:** `complete` or `input_required` from the core protocol, plus `task` from the tasks extension
- **Cacheable — also need** `ttlMs` **+** `cacheScope` **:** `server/discover`, `tools/list`, `prompts/list`, `resources/list`, `resources/templates/list`, `resources/read`
- **Must NOT carry them:** `prompts/get`, `tools/call`, `completion/complete`
- **This server's hints:** `ttlMs: 60000`, `cacheScope: "public"`

Error Codes

- `-32601` — removed method (deletion is physical: absence from the registry)
- `-32602` — incomplete `_meta` envelope
- `-32021` — missing client capability; data carries `requiredCapabilities`
- `-32022` — unsupported revision; data carries `supported` and `requested`
- `-32002`, `-32042` — retired. Never emit them; a *client* still accepts `-32002` from older servers

`-32020` … `-32099` is now reserved for the spec; `-32000` … `-32019` is legacy, and a new implementation should not use it. Resource-not-found, once `-32002`, is now just `-32602`.

Removed Methods

The removed requests answer `-32601`:

- `initialize`, `ping`, `logging/setLevel`
- `resources/subscribe`, `resources/unsubscribe`
- `tasks/result`, `tasks/list`

The removed notifications, `notifications/initialized` and `notifications/roots/list_changed`, are logged and dropped: nothing answers a notification.

Available Tools

- **Tool Name:** `key_word_search`
- **Description:** Searches for a specified keyword across all files in a project
- **Parameters:** keyword (required), directory (optional) — *omit the directory and the server asks over a round trip, then falls back to its working directory*
- **UI:** `_meta.ui.resourceUri` (canonical) *and* the deprecated `_meta["ui/resourceUri"]` → `ui://keyword-search/mcp-app.html`
- **No** `execution` **field** — `Tool.execution` / `taskSupport` was removed

Available Prompts

- **Prompt Name:** `search_keyword`
- **Description:** Creates a prompt, to search for a word using the key_word_search tool
- **Arguments:** keyword (required)

Available Resources

- **Javadoc:** the 75 HTML pages of the `spec` package, on `complete`
- **Example:** `javadoc/com/workshop/mcp/spec/Capability.`
- **MCP App:** `ui://keyword-search/mcp-app.html` served as `text/html;profile=mcp-app`
- **Pagination:** none — one response carries the whole list

Round-Trip Details

- **Embeddable methods:** `roots/list`, `sampling/createMessage`, `elicitation/create` — and nothing else. Sending any of the three *to* the server is `-32601`
- **Built by this server:** exactly one, `elicitation/create`. The other two are deprecated under SEP-2577
- **Shape:** `inputRequests` is a keyed map; each entry has `method` + optional `params`, no `jsonrpc`, no `id`
- **Key used here:** `search_directory`
- **Elicitation:** `mode: "form"`; the old `elicitationId` is gone
- `requestState` **:** opaque string, base64 JSON of `{stage}`, echoed back byte-exact; the keyword rides in the retried call's own arguments

Task Details

- **Trigger:** the server's choice — no `params.task` opt-in exists, and one that arrives MUST be ignored
- **Gate:** the client declared `io.modelcontextprotocol/tasks` on this request
- **Immediate reply:** `resultType: "task"` with `taskId`, status `working`
- **TTL / poll:** `ttlMs: 300000`, `pollIntervalMs: 1000`
- **Push:** none. `notifications/tasks` goes only to a client that subscribed to the task, and polling `tasks/get` is the default

Subscriptions

- **Method:** `subscriptions/listen`, one long-lived request carrying every notification
- **Filter fields:** `toolsListChanged`, `promptsListChanged`, `resourcesListChanged`, `resourceSubscriptions` — no slot for tasks in core; the tasks extension adds `taskIds`
- **Demultiplexed by:** `_meta["io.modelcontextprotocol/subscripti:` which holds the listen request's id
- **Honored here:** nothing — this server's lists are fixed (`listChanged: false`), so the stream closes at once

Omit the subscriptionId from the acknowledgment and the client hangs forever, with no error and no timeout.

Request Flow

1. `server/discover` — string id, no session created
2. `tools/list` — cacheable, so `ttlMs` + `cacheScope`
3. `tools/call` with a `directory` argument — one hop, nothing asked
4. `tools/call` without it → `input_required`, embedding `elicitation/create`
5. Retry with a new id, `inputResponses`, and the same `requestState`; declined or unaskable, and the working directory is the fallback
6. Same call from a tasks-capable client → `resultType: "task"`
7. `tasks/get` polled once a second until `completed`
8. `subscriptions/listen`, acknowledged and closed

Tool Execution Results

- **Search Keyword:** "record"
- **Search Directory:** from the call's `directory` argument, or the elicited directory, or the server's working directory
- **Files Matched:** 2 in `com/workshop/mcp/tools`, `KeyWordSearch.java` and `SearchContinuation.java` *(one small package, so the trace stays readable)*
- **Result Shape:** `resultType: "complete"` plus one text content block per file, `path, keyword_count=N`

This page is designed as a lightweight "presentation": open it full-screen and use your browser's zoom for big-room projection.

© 2026 MCP Workshop. Runtime information extracted from the captured trace.

MCP Workshop – Trace Log

Raw JSON-RPC messages exchanged between client and server on protocol revision 2026-07-28 — stateless, so every request carries its own _meta envelope.

Communication Trace

Toggle JSON Formatting All Received Sent

[2026-09-04 17:55:28.100] — Discovery — no handshake, no session, and a STRING request id

[2026-09-04 17:55:28.104] [API][RECEIVED] {"jsonrpc":"2.0","id":"server-discover-probe-1","method":"server/discover","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientInfo":{"name":"inspector-cli","version":"2.5.0"},"io.modelcontextprotocol/clientCapabilities":{"roots":{"listChanged":true},"extensions":{"io.modelcontextprotocol/tasks":{},"io.modelcontextprotocol/ui":{"mimeTypes":["text/html;profile=mcp-app"]}}}}}}

[2026-09-04 17:55:28.121] [API][SENT]: {"jsonrpc":"2.0","id":"server-discover-probe-1","result":{"resultType":"complete","supportedVersions":["2026-07-28"],"capabilities":{"completions":{},"prompts":{"listChanged":false},"resources":{"listChanged":false,"subscribe":false},"tools":{"listChanged":false},"extensions":{"io.modelcontextprotocol/tasks":{},"io.modelcontextprotocol/ui":{}}},"instructions":"Searches a project for a keyword. If no directory is supplied, the tool asks for one over a Multi Round-Trip Request, and searches its own working directory if the client offers nothing.","ttlMs":60000,"cacheScope":"public","_meta":{"io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:55:28.123] [API][SENT] server/discover – advertised [2026-07-28]; serverInfo travels in _meta, and there is no single protocolVersion

[2026-09-04 17:55:31.010] [API][RECEIVED] {"jsonrpc":"2.0","id":0,"method":"tools/list","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientInfo":{"name":"inspector-cli","version":"2.5.0"},"io.modelcontextprotocol/clientCapabilities":{"roots":{"listChanged":true},"extensions":{"io.modelcontextprotocol/tasks":{},"io.modelcontextprotocol/ui":{"mimeTypes":["text/html;profile=mcp-app"]}}}}}}

[2026-09-04 17:55:31.026] [API][SENT]: {"jsonrpc":"2.0","id":0,"result":{"resultType":"complete","tools":[{"name":"key_word_search","description":"Searches for a specified keyword across all files in a project. Returns the total count of matches and the absolute file paths of the files containing the keyword. Pass a directory to search it directly; omit it and the server asks for one over a Multi Round-Trip Request, falling back to its own working directory.","inputSchema":{"type":"object","properties":{"keyword":{"type":"string","description":"the keyword to search for in a file"},"directory":{"type":"string","description":"the absolute directory to search; omit it to have the server ask for one over a Multi Round-Trip Request and fall back to its working directory"}},"required":["keyword"]},"_meta":{"ui":{"resourceUri":"ui://keyword-search/mcp-app.html"},"ui/resourceUri":"ui://keyword-search/mcp-app.html"}}],"ttlMs":60000,"cacheScope":"public","_meta":{"io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:55:31.028] [API][SENT] tools/list – cacheable, so resultType + ttlMs + cacheScope are all required; no Tool.execution field any more

[2026-09-04 17:55:33.000] — Multi Round-Trip Request — the server asks by answering

[2026-09-04 17:55:33.004] [API][RECEIVED] {"jsonrpc":"2.0","id":2,"method":"tools/call","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientInfo":{"name":"inspector-cli","version":"2.5.0"},"io.modelcontextprotocol/clientCapabilities":{"roots":{"listChanged":true},"elicitation":{"form":{},"url":{}}}},"name":"key_word_search","arguments":{"keyword":"record"}}}

[2026-09-04 17:55:33.012] [API][SENT] tools/call – input_required, embedding elicitation/create

[2026-09-04 17:55:33.014] [API][SENT]: {"jsonrpc":"2.0","id":2,"result":{"resultType":"input_required","inputRequests":{"search_directory":{"method":"elicitation/create","params":{"mode":"form","message":"Pick a directory for the keyword-search tool to search in.","requestedSchema":{"type":"object","properties":{"directory":{"type":"string","title":"Search directory","description":"Absolute path to the directory the keyword search should look in"}},"required":["directory"]}}},"requestState":"eyJzdGFnZSI6ImRpcmVjdG9yeSJ9","_meta":{"io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:55:33.015] [API][SENT] inputRequests is a keyed MAP; the embedded request has no jsonrpc and no id. requestState is opaque base64 of {"stage":"directory"}; the keyword rides in the retry's own arguments

[2026-09-04 17:55:35.220] [API][RECEIVED] {"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientInfo":{"name":"inspector-cli","version":"2.5.0"},"io.modelcontextprotocol/clientCapabilities":{"roots":{"listChanged":true},"elicitation":{"form":{},"url":{}}}},"name":"key_word_search","arguments":{"keyword":"record"},"requestState":"eyJzdGFnZSI6ImRpcmVjdG9yeSJ9","inputResponses":{"search_directory":{"action":"accept","content":{"directory":"/Users/davidparry/code/github/mcp-servers/agent-mcp-workshop/src/main/java/com/workshop/mcp/tools"}}}}}

[2026-09-04 17:55:35.222] [API][RECEIVED] tools/call retry – BRAND NEW id 3, requestState echoed byte-exact, elicited directory=/Users/davidparry/code/github/mcp-servers/agent-mcp-workshop/src/main/java/com/workshop/mcp/tools

[2026-09-04 17:55:35.223] the client still declares "roots":{"listChanged":true} in that envelope, and it is right to – roots is deprecated, not removed, so clients keep declaring it. This server does not model the key and ignores it

[2026-09-04 17:55:35.284] [API][SENT]: {"jsonrpc":"2.0","id":3,"result":{"resultType":"complete","content":[{"text":"/Users/davidparry/.../mcp/tools/KeyWordSearch.java, keyword_count=1","type":"text"},{"text":"/Users/davidparry/.../mcp/tools/SearchContinuation.java, keyword_count=1","type":"text"}],"isError":false,"_meta":{"io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:55:38.000] — One hop — the same tool with an explicit directory argument never has to ask at all

[2026-09-04 17:55:38.004] [API][RECEIVED] {"jsonrpc":"2.0","id":4,"method":"tools/call","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientInfo":{"name":"inspector-cli","version":"2.5.0"},"io.modelcontextprotocol/clientCapabilities":{"roots":{"listChanged":true},"elicitation":{"form":{},"url":{}}}},"name":"key_word_search","arguments":{"keyword":"record","directory":"/Users/davidparry/code/github/mcp-servers/agent-mcp-workshop/src/main/java/com/workshop/mcp/tools"}}}

[2026-09-04 17:55:38.009] [API][SENT] tools/call – the directory argument settles it outright, so no input_required is needed. Passing directories as tool parameters is the migration the spec names in place of roots

[2026-09-04 17:55:38.070] [API][SENT]: {"jsonrpc":"2.0","id":4,"result":{"resultType":"complete","content":[{"text":"/Users/davidparry/.../mcp/tools/KeyWordSearch.java, keyword_count=1","type":"text"},{"text":"/Users/davidparry/.../mcp/tools/SearchContinuation.java, keyword_count=1","type":"text"}],"isError":false,"_meta":{"io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:55:44.000] — Fallback — the user declines the form, and the call still returns a result

[2026-09-04 17:55:44.100] [API][RECEIVED] {"jsonrpc":"2.0","id":5,"method":"tools/call","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientInfo":{"name":"inspector-cli","version":"2.5.0"},"io.modelcontextprotocol/clientCapabilities":{"roots":{"listChanged":true},"elicitation":{"form":{},"url":{}}}},"name":"key_word_search","arguments":{"keyword":"record"},"requestState":"eyJzdGFnZSI6ImRpcmVjdG9yeSJ9","inputResponses":{"search_directory":{"action":"decline"}}}}

[2026-09-04 17:55:44.104] [API][SENT] tools/call – no directory to search, using the working directory /Users/davidparry/code/github/mcp-servers/agent-mcp-workshop/inspector

[2026-09-04 17:55:44.166] [API][SENT]: {"jsonrpc":"2.0","id":5,"result":{"resultType":"complete","content":[{"text":"/Users/davidparry/.../inspector/modern-probe.sh, keyword_count=1","type":"text"},{"text":"/Users/davidparry/.../inspector/modern-mrtr.sh, keyword_count=5","type":"text"},{"text":"/Users/davidparry/.../inspector/modern-tasks.sh, keyword_count=1","type":"text"}],"isError":false,"_meta":{"io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:55:44.168] the asking has run out, so the server searches where it was started – the same place a shell command with no path argument would look. The server was started in inspector/, so the matches are the probe scripts themselves

[2026-09-04 17:56:06.270] — Task handle — same call, but this client declared the tasks extension

[2026-09-04 17:56:06.272] [API][RECEIVED] {"jsonrpc":"2.0","id":1,"method":"tools/call","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientInfo":{"name":"inspector-cli","version":"2.5.0"},"io.modelcontextprotocol/clientCapabilities":{"roots":{"listChanged":true},"extensions":{"io.modelcontextprotocol/tasks":{},"io.modelcontextprotocol/ui":{"mimeTypes":["text/html;profile=mcp-app"]}}}},"name":"key_word_search","arguments":{"keyword":"record","directory":"/Users/davidparry/code/github/mcp-servers/agent-mcp-workshop/src/main/java/com/workshop/mcp/tools"}}}

[2026-09-04 17:56:06.275] [API][SENT]: tools/call – created taskId=95a30b18-0626-4160-acf6-3f3ac215c19f status=working. There is no params.task opt-in any more; this is the server's decision

[2026-09-04 17:56:06.276] this envelope declares tasks but not elicitation, so the directory has to arrive as an argument. Drop it and the server still comes back first – resultType holds one value – and the question then arrives as the task's own input_required status, answered with tasks/update

[2026-09-04 17:56:06.279] [API][SENT]: {"jsonrpc":"2.0","id":1,"result":{"resultType":"task","taskId":"95a30b18-0626-4160-acf6-3f3ac215c19f","status":"working","statusMessage":"The operation is now in progress.","createdAt":"2026-09-04T17:56:06.276660Z","lastUpdatedAt":"2026-09-04T17:56:06.276660Z","ttlMs":300000,"pollIntervalMs":1000},"_meta":{"io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:56:06.281] [TASK 95a30b18-0626-4160-acf6-3f3ac215c19f] background tool execution started for tool=key_word_search – no status notification follows; this server pushes none, so the client polls

[2026-09-04 17:56:07.290] [API][RECEIVED] {"jsonrpc":"2.0","id":"inspector-ext-1","method":"tasks/get","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientCapabilities":{"extensions":{"io.modelcontextprotocol/tasks":{}}}},"taskId":"95a30b18-0626-4160-acf6-3f3ac215c19f"}}

[2026-09-04 17:56:07.292] [API][SENT]: {"jsonrpc":"2.0","id":"inspector-ext-1","result":{"resultType":"complete","taskId":"95a30b18-0626-4160-acf6-3f3ac215c19f","status":"working","statusMessage":"The operation is now in progress.","createdAt":"2026-09-04T17:56:06.276660Z","lastUpdatedAt":"2026-09-04T17:56:06.276660Z","ttlMs":300000,"pollIntervalMs":1000,"_meta":{"io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:56:09.350] [API][SENT] tasks/get inspector-ext-2 and inspector-ext-3 – still working; the client polls once a second, as pollIntervalMs suggests

[2026-09-04 17:56:10.292] [TASK 95a30b18-0626-4160-acf6-3f3ac215c19f] tool completed, transitioning to completed

[2026-09-04 17:56:10.400] [API][RECEIVED] {"jsonrpc":"2.0","id":"inspector-ext-4","method":"tasks/get","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientCapabilities":{"extensions":{"io.modelcontextprotocol/tasks":{}}}},"taskId":"95a30b18-0626-4160-acf6-3f3ac215c19f"}}

[2026-09-04 17:56:10.403] [API][SENT]: {"jsonrpc":"2.0","id":"inspector-ext-4","result":{"resultType":"complete","taskId":"95a30b18-0626-4160-acf6-3f3ac215c19f","status":"completed","statusMessage":"The operation completed successfully.","createdAt":"2026-09-04T17:56:06.276660Z","lastUpdatedAt":"2026-09-04T17:56:10.294781Z","ttlMs":300000,"pollIntervalMs":1000,"result":{"resultType":"complete","content":[{"text":"/Users/davidparry/.../mcp/tools/KeyWordSearch.java, keyword_count=1","type":"text"},{"text":"/Users/davidparry/.../mcp/tools/SearchContinuation.java, keyword_count=1","type":"text"}],"isError":false},"_meta":{"io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:56:11.000] — subscriptions/listen — one request carries every server-to-client notification

[2026-09-04 17:56:11.004] [API][RECEIVED] {"jsonrpc":"2.0","id":"listen:0","method":"subscriptions/listen","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientCapabilities":{"roots":{"listChanged":true}}},"notifications":{"toolsListChanged":true,"promptsListChanged":true,"resourcesListChanged":true,"resourceSubscriptions":["ui://keyword-search/mcp-app.html"]}}}

[2026-09-04 17:56:11.008] [API][SENT] subscriptions/listen – acknowledging id=listen:0, honored=nothing. This server advertises listChanged=false, so the intersection is empty

[2026-09-04 17:56:11.010] [API][SENT]: {"jsonrpc":"2.0","method":"notifications/subscriptions/acknowledged","params":{"_meta":{"io.modelcontextprotocol/subscriptionId":"listen:0"}},"notifications":{}}}

[2026-09-04 17:56:11.011] [API][SENT] the subscriptionId in that _meta is mandatory – omit it and the client waits forever, with no error and no timeout

[2026-09-04 17:56:11.013] [API][SENT]: {"jsonrpc":"2.0","id":"listen:0","result":{"resultType":"complete","_meta":{"io.modelcontextprotocol/subscriptionId":"listen:0","io.modelcontextprotocol/serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}}

[2026-09-04 17:56:14.000] — Rejections — deletion in this revision is physical, and the envelope is validated per request

[2026-09-04 17:56:14.004] [API][RECEIVED] {"jsonrpc":"2.0","id":4,"method":"initialize","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-28","io.modelcontextprotocol/clientCapabilities":{"roots":{"listChanged":true}}}}}

[2026-09-04 17:56:14.006] [API][SENT]: {"jsonrpc":"2.0","id":4,"error":{"code":-32601,"message":"Method not found: initialize. This server only supports protocol versions without the initialize handshake: 2026-07-28","data":{"supported":["2026-07-28"]}}}

[2026-09-04 17:56:14.020] [API][RECEIVED] {"jsonrpc":"2.0","id":5,"method":"tools/list","params":{"_meta":{"io.modelcontextprotocol/protocolVersion":"2025-11-25","io.modelcontextprotocol/clientCapabilities":{}}}}

[2026-09-04 17:56:14.022] [API][SENT]: {"jsonrpc":"2.0","id":5,"error":{"code":-32022,"message":"Unsupported protocol version","data":{"supported":["2026-07-28"],"requested":"2025-11-25"}}}

[2026-09-04 17:56:14.030] [API][RECEIVED] {"jsonrpc":"2.0","id":6,"method":"tools/list","params":{}}

[2026-09-04 17:56:14.032] [API][SENT]: {"jsonrpc":"2.0","id":6,"error":{"code":-32602,"message":"Requests must carry io.modelcontextprotocol/protocolVersion and io.modelcontextprotocol/clientCapabilities in params._meta"}}

[2026-09-04 17:56:16.615] Stopping and closing resources

Trace assembled from inspector/modern-probe.sh and inspector/modern-mrtr.sh against the workshop server. The search root is one small package, so the tool matches only two files — the payload is deliberately compact so the protocol flow is the focus. Note also that the plain and the task-bearing tools/call are the same request; only the client's declared capabilities differ.

This trace shows the complete stateless lifecycle — discovery, cacheable reads, a Multi Round-Trip Request, a task handle, a subscription that closes at once, and the rejections. Color coding: Blue = Received messages, Green = Sent messages, Red = System events

© 2026 MCP Workshop. Raw trace data from MCP communication on revision 2026-07-28.



DAVID PARRY

Principal Architect

x.com

YouTube

LinkedIn

Github

Discord

Email

https://github.com/David-Parry/agent-mcp-workshop/tree/complete

© 2026 MCP Workshop. Contact page.